

Module OPSC6311: Part 1 (Planning & Design)Student Number: **ST10345327**Name: **Olebogeng Phawe****1. Introduction**

This document presents the planning and design phase of the Personal Budget Tracker app, SaveSmart, built for Android using Kotlin. It covers the user interface design, including mock-ups for all ten screens created in Figma Make, a purpose and element description for each screen, and a navigation diagram showing how users move between screens. It also contains a detailed list of all 24 functional requirements, a design decisions section, and a two-week project Gantt chart with milestones.

SaveSmart helps users track their spending habits, set budget goals, and stay motivated through a gamification system of points, badges, and level progression. All design decisions are directly informed by the research conducted in the accompanying Research document particularly the gap analysis showing no existing app offers comprehensive gamification, and the YNAB API insight on milliunit currency storage.

2. App Overview — SaveSmart**2.1 App Name and Concept**

The app is named SaveSmart. The tagline is Smart Saving Made Simple. The app icon features a credit card design reflecting the financial nature of the app, created as part of the UI design process. The name communicates both the core function smart budgeting and the goal of helping users develop better saving habits over time.

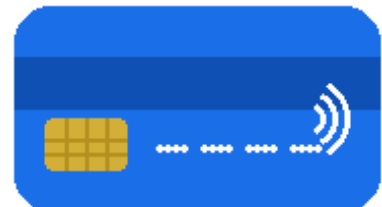


FIGURE 1: SAVESMART APP ICON

2.2 Target Users

- Young adults aged 18–35 who are new to managing their own finances
- Students who need to track limited monthly budgets
- Working professionals who want to develop better saving habits
- Any user who finds traditional budgeting apps too dry or tedious

2.3 Core Technology Stack

- **Platform:** Android (minimum SDK 26 - Android 8.0 Oreo)
- **Language:** Kotlin
- **Database:** Room (SQLite) local, offline-first storage
- **Architecture:** MVVM (Model-View-ViewModel) with LiveData
- **UI:** XML layouts with Material Design 3 components
- **Charts:** MPAndroidChart library for spending graphs
- **Authentication:** Username + hashed password stored in local Room DB
- **Currency storage:** Milliunit convention (Long integers) inspired by the YNAB API data model (YNAB API, 2024) amounts stored as e.g. 12500 representing R12.50, converted to display values on screen to avoid floating point rounding errors

2.4 Gamification Design

The rewards system is designed to feel achievable rather than overwhelming. Points are awarded for the following actions:

- **Logging any expense:** +5 points
- **Staying under a category spending limit for the full month:** +25 points
- **Logging expenses 7 days in a row:** +50 points
- **Achieving the monthly budget goal:** +100 points
- **Logging expenses 30 consecutive days:** +200 points

Badges are unlocked for reaching milestones and displayed in a 2-column grid. Earned badges appear in full colour while locked badges are faded, motivating the user to keep progressing.

3. Detailed Feature Requirements

3.1 Authentication

<i>ID</i>	<i>Feature</i>	<i>Description</i>	<i>Source</i>
R01	User Registration	User enters a username and password to create an account. Password is confirmed on entry and stored securely using SHA-256 hashing in the local Room database.	POE Part 2 requirement
R02	User Login	User logs in using their username and password. The app validates the credentials against the Room database. An error message is shown for invalid input. The session is kept active until the user logs out.	POE Part 2 requirement
R03	Input Validation	All login and registration fields must be validated. Empty fields, mismatched passwords, and duplicate usernames must show clear, user-friendly error messages without crashing the app.	POE Part 2 requirement
R04	Logout	User can log out from the app. Session data is cleared from SharedPreferences and the user is returned to the Login screen.	POE Part 2 requirement

3.2 Expense Categories

<i>ID</i>	<i>Feature</i>	<i>Description</i>	<i>Source</i>
R05	Create Category	User can create a named expense category and assign it a colour. Categories are stored in the Room database and linked to expense entries.	POE Part 2 requirement
R06	Edit Category	User can edit the name, colour, and spending limits of any existing category.	POE Part 2 requirement
R07	Delete Category	User can delete a category. A confirmation dialog is shown before deletion. Associated expenses are updated to reflect the removed category.	POE Part 2 requirement

3.3 Expense Entries

<i>ID</i>	<i>Feature</i>	<i>Description</i>	<i>Source</i>
R08	Add Expense Entry	User creates an expense entry with the following fields: date, start time, end time, description, and category. All fields except description are required. Amount is stored as milliunit Long integers (e.g. R12.50 = 12500) to avoid floating point errors.	POE Part 2 requirement
R09	Receipt Photo Attachment	User can optionally attach a photo to any expense entry by taking a new photo with the camera or selecting an existing image from the gallery. The file path is stored in the Room database and a thumbnail is shown in the expense list.	POE Part 2 requirement
R10	View Expense List	User can view a scrollable list of all expense entries filtered by a user-selectable period using date pickers. Each row shows the description, category, date, start and end times, and amount. A receipt icon is shown if a photo is attached.	POE Part 2 requirement
R11	View Receipt Photo	Tapping the receipt icon on any expense entry in the list opens the attached photo in full screen. This is accessible directly from the expense list without navigating to a separate screen.	POE Part 2 requirement
R12	Edit / Delete Expense	User can tap any expense entry to edit its details or delete it. A confirmation dialog is shown before deletion to prevent accidental removal.	Own feature
R13	Input Validation	All expense entry fields must be validated. The app must handle invalid input (e.g. empty required fields, invalid date/time values) gracefully with clear error messages and without crashing.	POE Part 2 requirement

3.4 Budget Goals

<i>ID</i>	<i>Feature</i>	<i>Description</i>	<i>Source</i>
R14	Set Min/Max Monthly Goals	User can set both a minimum spending goal and a maximum spending goal for each month. Both values are stored per category per month in the Room database as milliunits. The dashboard visually indicates whether spending is below the minimum, within the goal range, or above the maximum.	POE Part 2 requirement
R15	Budget Progress Dashboard	The home screen shows a summary card for total monthly budget progress and individual cards per category, each with a colour-coded progress bar and status label (Good / Close / Over!) based on the min/max goals.	POE Part 2 requirement
R16	Overspend Highlighting	Any category where spending exceeds the maximum goal is highlighted in red with an 'Over!' status badge. Categories approaching the maximum show an amber 'Close' badge. Categories below the minimum goal are also flagged to encourage spending if needed.	POE Part 2 requirement

3.5 Reports and Graphs

<i>ID</i>	<i>Feature</i>	<i>Description</i>	<i>Source</i>
R17	Category Spending Report	User views the total amount spent per category for a user-selectable time period. Displayed as a donut/pie chart and a ranked list showing the category name, total amount, and percentage of total spending.	POE Part 2 requirement
R18	Daily Spending Graph	A bar chart (built using MPAndroidChart) shows daily spending totals over a user-selected period. Bars that exceed the daily budget limit are shown in red. A dashed horizontal line marks the daily budget threshold.	POE Part 2 requirement

3.6 Gamification

<i>ID</i>	<i>Feature</i>	<i>Description</i>	<i>Source</i>
R19	Points System	Users earn points for positive financial behaviours: +5 pts per expense logged, +25 pts per category goal met for the month, +50 pts for a 7-day logging streak, +100 pts for achieving the monthly budget goal. Points accumulate and are stored in the Room database.	POE Part 2 requirement — Final POE
R20	Badges	Milestone badges are permanently awarded when specific goals are reached: First Save, 7-Day Streak, 30-Day Streak, Budget Master (under budget 3 months), Quick Logger (10 expenses in one day), Zero Spend Day, and Goal Crusher (all categories within limits).	POE Part 2 requirement
R21	Level Progression	Points unlock levels: Level 1 (0–99 pts), Level 2 (100–249 pts), Level 3 (250–499 pts), Level 4 (500–999 pts), Level 5 (1000+ pts). The current level and progress to the next level are displayed on the Rewards screen.	own feature
R22	Leaderboard	A leaderboard tab on the Rewards screen ranks all registered users by their total lifetime points. Each entry shows the username, level badge, and points total, drawn from the local Room database.	own feature

3.7 Technical Requirements

<i>ID</i>	<i>Feature</i>	<i>Description</i>	<i>Source</i>
T01	Language & Platform	The app must be built using Kotlin for Android. Minimum SDK 26 (Android 8.0 Oreo). Architecture pattern: MVVM with LiveData and ViewModel.	POE Part 2 requirement
T02	Local Database	All app data must be stored in a local Room (SQLite) database on the device. No cloud storage or internet connection is required for the app to function.	POE Part 2 requirement
T03	GitHub Repository	The project must be hosted on GitHub with a clear README file that explains the app, how to build it, and how to run the tests. Regular, meaningful commits must be made throughout development to show progress.	POE Part 2 requirement
T04	GitHub Actions CI	A GitHub Actions workflow must be configured to automatically build the project and run all unit tests whenever code is pushed to the repository. The workflow file must be included in the repository.	POE Part 2 requirement
T05	Unit Testing	Unit tests must be written for all Room DAO queries, authentication logic, milliunit currency calculations, and budget goal comparisons. Tests must pass in the GitHub Actions CI pipeline.	POE Part 2 requirement
T06	Commented Code & Logging	All Kotlin classes, functions, and complex logic must include clear comments explaining what the code does. Android Log statements must be used throughout for debugging, using appropriate log levels (Log.d, Log.e, Log.w).	POE Part 2 requirement
T07	Built APK	A working compiled APK file of the final app must be submitted alongside the source code. The APK must install and run correctly on an Android device or emulator.	POE Part 2 requirement

T08	Demonstration Video	A video demonstrating all app features must be submitted with a voice-over explanation. The video must be compressed to a reasonable file size. It must show all core features working on a real device or emulator.	POE Part 2 requirement
T09	User-Friendly UI	The app must handle all invalid input gracefully without crashing. Error messages must be clear and helpful. The UI must follow Material Design 3 guidelines and match the Figma Make prototype design.	POE Part 2 requirement
T10	Milliunit Currency Storage	All monetary amounts are stored as Long integers in milliunits (1 000 milliunits = R1.00). For example R12.50 is stored as 12 500. Converted to Rand display values only when shown on screen. This prevents floating point rounding errors in budget calculations. Pattern adopted from the YNAB API (YNAB API, 2024).	YNAB API research

3.8 Onboarding

<i>ID</i>	<i>Feature</i>	<i>Description</i>	<i>Source</i>
R23	Interactive Onboarding Flow	A 3-step guided setup screen shown once after first registration. Uses emoji-illustrated multiple-choice questions to help the user: (1) set their monthly min/max budget goals, (2) create their first spending category, (3) understand the points and badges system. A skip option is available on every step. Inspired by YNAB's onboarding UX	YNAB research

4. User Interface Design

4.1 Screen Summary Table

The following table summarises all ten screens of the SaveSmart app, their purpose, and key UI elements.

#	Screen Name	Purpose	Key UI Elements
1	Login Screen	Entry point to the app. Allows existing users to authenticate with username and password.	Logo/app name, username text field, password field (masked), Login button, Register link, Forgot Password link
2	Register Screen	Allows new users to create an account by entering their name, username, and password.	Full name field, username field, password field, confirm password field, Register button, Back to Login link
3	Onboarding Flow	A 3-step interactive welcome flow shown once after registration. Inspired by YNAB's onboarding UX uses emoji-illustrated multiple-choice questions to help the user set their monthly budget goal, create their first category, and understand the points/badges system before reaching the dashboard.	Step progress indicator, illustrated emoji question cards, multiple-choice answer buttons, Skip option, animated transitions between steps
4	Dashboard (Home)	Main overview screen. Displays the current monthly budget progress and per-category status. Overspending categories are highlighted in red.	Greeting header with month, total budget progress bar, category cards with colour-coded status badges (On Track / Close / Over!), bottom navigation bar
5	Add Expense	Allows the user to log a new expense entry with all required fields on a single screen.	Amount field (R), date picker, start time picker, end time picker, description field, category selector chips, optional receipt photo picker (camera/gallery), Save button

6	Expense List	Shows a scrollable, filterable list of all expenses within a user-selected date range. Receipt indicator shown per entry.	Date range pickers (From / To), scrollable list of expense cards (name, category, date, amount), receipt icon indicator on entries with photos
7	Category Report	Displays total spending per category for the selected period, with a pie chart for visual breakdown.	Date range selector, pie/donut chart coloured by category, list of categories with total amounts and colour indicators
8	Spending Graph	Shows a daily bar chart of spending over the selected period. Bars exceeding the daily budget limit are highlighted red.	Date range selector, vertical bar chart (MPAndroidChart), colour legend (normal / over limit), dashed daily limit line
9	Rewards Screen	Gamification hub. Shows the user's points, level, earned badges, and progress towards locked badges.	User level badge, points counter, featured badge display, 2-column badge grid (earned = full colour, locked = faded), leaderboard tab
10	Categories Screen	Allows the user to create and manage expense categories with optional minimum and maximum monthly spending goals per category.	Add Category button, list of existing categories with colour indicator, min/max goal amounts, progress bar, edit/delete actions per category

4.2 Screen Mockups

The following mockups are actual screenshots from the SaveSmart prototype built in Figma Make (Figma, 2026). The design follows a clean, minimalist light theme

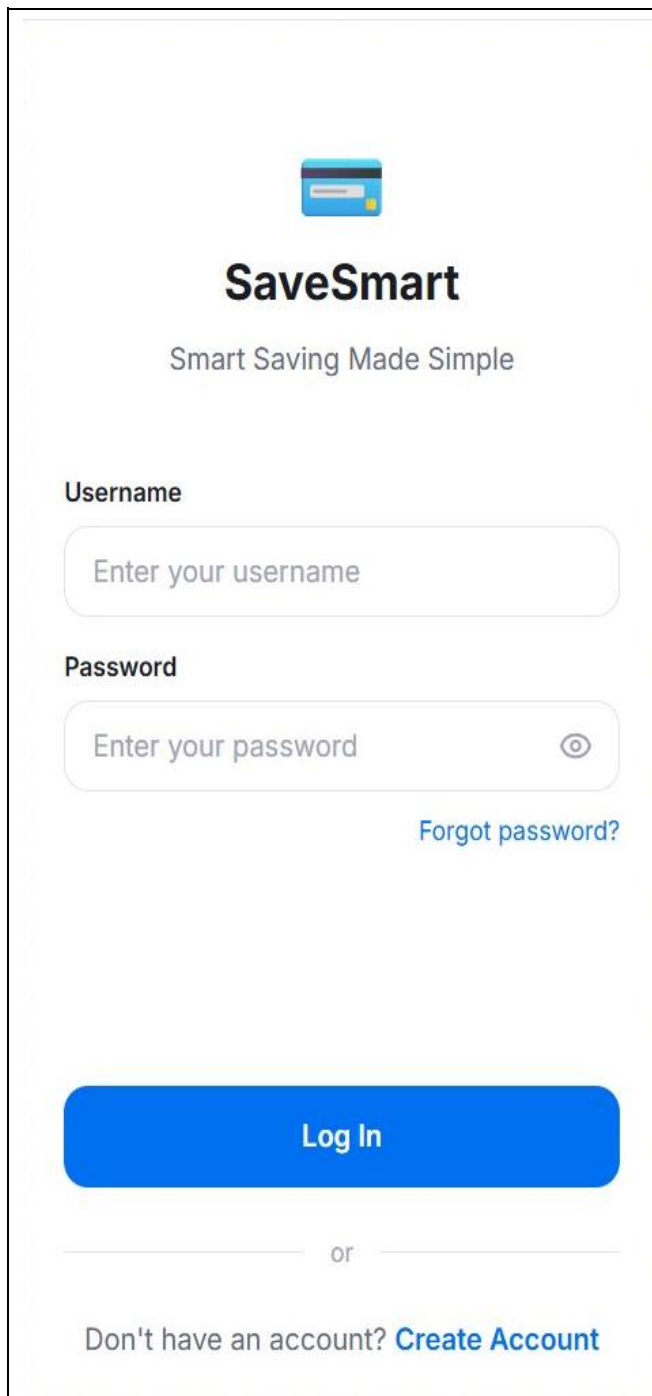


Figure 1: (Login Screen) SaveSmart branding with credit card icon and Smart Saving Made Simple tagline

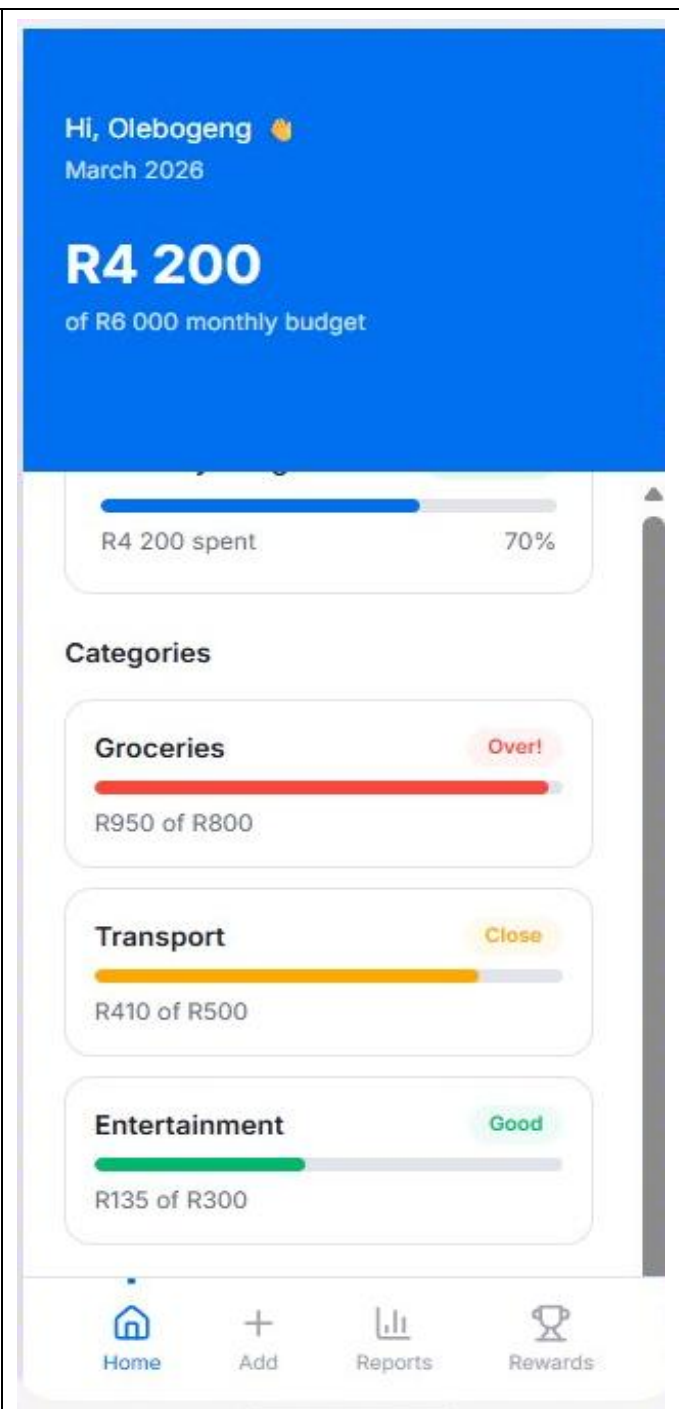


Figure 2: (Dashboard) blue header, R4 200 monthly spend, colour-coded category cards (Over!/Close/Good)

Figure 3: (Add Expense) amount input, description, date picker, category chips, receipt photo section

Figure 4: (Expense List) date filter, SA merchants (Shoprite, Pick n Pay, Bolt), receipt icons

Figure 5: (Rewards Screen) Level 4 Saver purple banner, Budget Master badge, earned/locked badge grid

5. Navigation Design

5.1 Navigation Structure Overview

The app uses a bottom navigation bar as the primary navigation mechanism for the four main sections: Home (Dashboard), Add Expense, Reports, and Rewards. Secondary screens (Register, Categories, spending sub-views) are reached via contextual buttons or settings links within main screens.

5.2 Navigation Flow Description

Authentication Flow

- App launches => Login Screen
- New user taps 'Register' => Register Screen => on success => Dashboard
- Returning user enters credentials => Login => Dashboard

Main Navigation (Bottom Nav Bar)

- Home tab => Dashboard Screen (default after login)
- Add tab => Add Expense Screen => on save => returns to Dashboard
- Reports tab => Expense List Screen
- Rewards tab => Rewards Screen

Reports Sub-Navigation

- Expense List => tap 'By Category' button => Category Report Screen
- Expense List => tap 'Graph View' button => Spending Graph Screen
- Expense entry with receipt icon => tap to open full-screen receipt photo

Settings / Management Navigation

- Dashboard => tap Settings icon => Categories Screen
- Categories Screen => tap '+ Add Category' => Add/Edit Category Dialog
- Dashboard => tap budget goal edit icon => Set Monthly Budget Goal Dialog

5.3 Navigation Diagram

The diagram below illustrates all screen-to-screen navigation paths. Solid arrows indicate primary navigation flows triggered by the bottom navigation bar or main action buttons. Dashed arrows indicate secondary or settings-level navigation.

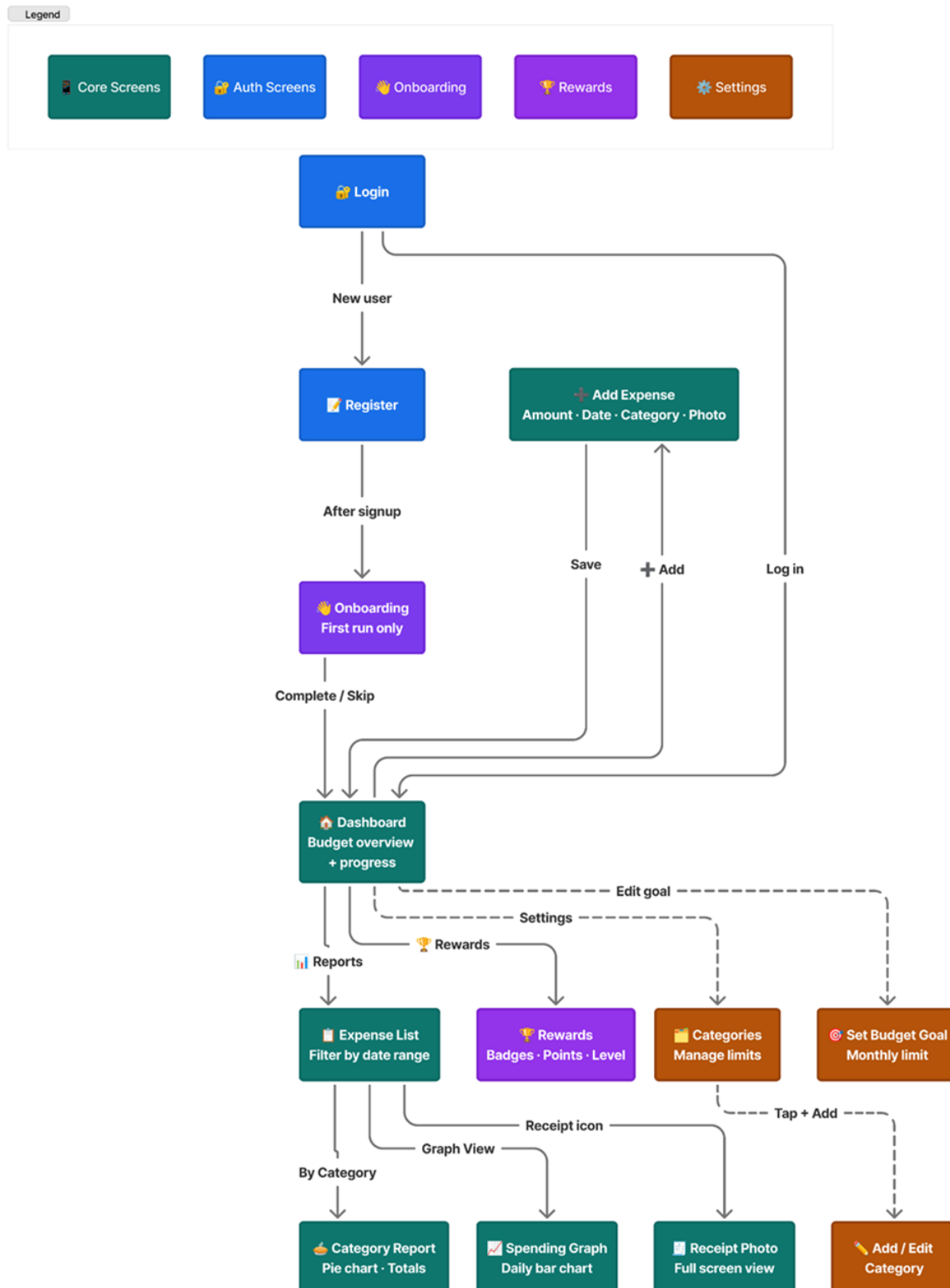


Figure 11: SaveSmart navigation diagram — all screen-to-screen flows

6. Design Decisions

6.1 Colour Scheme

The app uses a clean light theme with a vivid blue accent colour to convey trust and clarity. Semantic colours are used consistently throughout the app:

- Blue (#1A6FE8) primary accent, buttons, active navigation, progress bars
- Green (#16A34A) under budget / on track / earned badge
- Amber (#F59E0B) approaching budget limit (warning / Close status)
- Red (#DC2626) overspent / over limit / Over! status alert
- Purple (#7C3AED) gamification / rewards / badges
- White (FFFFFF) card backgrounds and screen surfaces
- Light grey (#F5F5F7) screen background

6.2 Typography and Layout

The following layout and typography decisions were applied consistently across all screens:

- **Font:** Inter / system sans-serif, clean and modern, easy to read on small screens
- **Screen padding:** 16dp on all sides (provides breathing room without wasting) space
- **Card radius:** 16dp - soft rounded corners for a modern, friendly feel
- **Button height:** 54dp, radius 12dp (large enough for easy tapping)
- **Bottom navigation bar:** 4 tabs (Home, Add, Reports, Rewards) – (thumb-friendly) access
- Cards with 1dp border and no drop shadow (clean, flat, minimal style)
- Material Design 3 guidelines followed for all component sizing and spacing

6.3 Gamification Design

The rewards system was designed to motivate users without feeling overwhelming. Key design decisions include:

- Points awarded immediately after each qualifying action, instant positive feedback
- Badges displayed in a 2-column grid, earned badges in full colour, locked badges faded
- Level progression shown with a progress bar on the Rewards screen
- Leaderboard tab shows all users ranked by total points, adds social motivation
- Points system: +5 per expense logged, +25 per category goal met, +50 for 7-day streak, +100 for monthly goal achieved

7. Project Plan — Gantt Chart

7.1 Project Timeline Overview

The project spans six weeks from Monday 17 March 2026 to Tuesday 28 April 2026. Part 1 research and planning, runs from 17 to 24 March 2026.

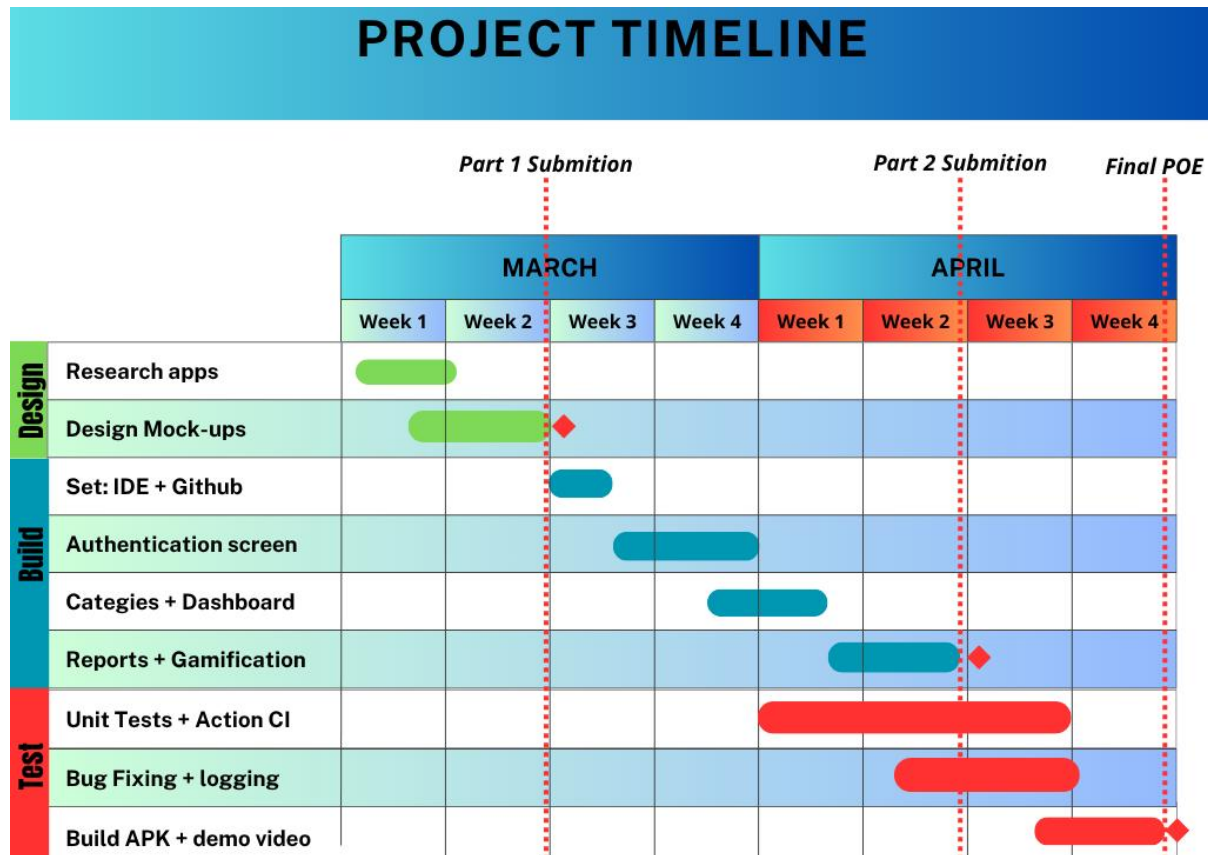


Figure 12: SaveSmart full project Gantt chart — 17 March to 28 April 2026

7.2 Milestones

Ref	Symbol	Description	Target Date
M1	◆ Red diamond	Part 1 research and planning documents submitted	March Week 2 (24 March 2026)
M2	◆ Red diamond	Part 2 room DB schema, GitHub repo, and README in place	April Week 2 (10 April 2026)
M3	◆ Red diamond	Final POE core features complete, auth, categories, add expense with start/end times, expense list	April Week 4 (28 April 2026)

8. Conclusion

This planning and design document provides the complete foundation for SaveSmart, a Personal Budget Tracker Android app built in Kotlin. The document covers 33 detailed requirements across eight categories, authentication, expense categories, expense entries (with date, start time, end time, and receipt photos), min/max budget goals, reports, gamification, technical requirements, and onboarding. The user interface was designed in Figma Make using a clean minimalist light theme. All 10 screens are documented with mockups and purpose descriptions. The navigation diagram shows all primary and secondary screen flows. The Gantt chart covers a two-week development plan with six milestones, ending with a built APK, commented code, GitHub Actions CI, and a recorded demonstration video. The gamification system points, badges, and level progression, addresses the main gap found across all three researched apps.

9. References

- Figma (2026) SaveSmart Mobile App UI — prototype built using Figma Make. Figma Inc. Available at: <https://www.figma.com/make/9DHUnU1yesFL3NrcN5AFAZ/SaveSmart-Mobile-App-UI> (Accessed: 24 March 2026).
- Google LLC (2024) Bottom navigation — Material Design 3. Material Design. Available at: <https://m3.material.io/components/navigation-bar/overview> (Accessed: 16 March 2026).
- Google LLC (2024) Guide to app architecture (MVVM). Android Developers. Available at: <https://developer.android.com/topic/architecture> (Accessed: 16 March 2026).
- Google LLC (2024) Room persistence library. Android Developers. Available at: <https://developer.android.com/training/data-storage/room> (Accessed: 16 March 2026).
- Jahoda, P. (2024) MPAndroidChart - a powerful chart library for Android. GitHub. Available at: <https://github.com/PhilJay/MPAndroidChart> (Accessed: 16 March 2026).
- Patel, N. (2023) YNAB review: a deep dive into zero-based budgeting. Investopedia. Available at: <https://www.investopedia.com/ynab-review-5094415> (Accessed: 16 March 2026).
- YNAB API (2024) YNAB API v1 documentation: milliunits and data model. You Need A Budget LLC. Available at: <https://api.ynab.com/> (Accessed: 16 March 2026).